

Day 7: Collections Framework – Part 2 (16-June-2026, Tuesday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 7, learners will be able to:

- Use `HashMap` and `TreeMap` to store key-value pairs and understand their ordering, null handling, and performance.
- Use `HashSet` and `TreeSet` to store unique elements and compare their iteration order.
- Implement `Comparable` (natural ordering) and `Comparator` (custom ordering) for objects.
- Solve LeetCode: *Group Anagrams* (`HashMap`), *Top K Frequent Elements* (`HashMap` + `Heap`/`PriorityQueue`), *Majority Element* (`Boyer-Moore Voting` / `HashMap`).
- Solve HackerRank: *Java Map* (phone book), *Java Hashset* (count unique pairs).
- Design a **Fraud Detection & Transaction Monitoring System** using maps and sets.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 6 & Day 7 Overview – Maps and Sets	15 min
9:15 – 10:00	<code>HashMap</code> – Internal Structure, Hashing, Methods, Performance	45 min
10:00 – 10:30	<code>TreeMap</code> – Red-Black Tree, <code>SortedMap</code> , <code>NavigableMap</code>	30 min
10:30 – 10:45	Tea Break	15 min
10:45 – 11:15	<code>HashSet</code> – Backed by <code>HashMap</code> , Uniqueness via <code>hashCode/equals</code>	30 min
11:15 – 11:45	<code>TreeSet</code> – Sorted Unique Elements, <code>NavigableSet</code>	30 min
11:45 – 12:15	<code>Comparable</code> Interface – Natural Ordering (<code>String</code> , <code>Integer</code> , <code>Custom</code>)	30 min

12:15	Custom)	
12:15 – 12:30	Comparator Interface – Custom Sorting Multiple Ways	15 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:00	LeetCode: Group Anagrams & Top K Frequent Elements (Approaches)	30 min
2:00 – 2:30	LeetCode: Majority Element (Boyer-Moore & HashMap solutions)	30 min
2:30 – 2:45	HackerRank: Java Map & Java Hashset Walkthrough	15 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:45	Corporate Use Case: Fraud Detection & Transaction Monitoring	45 min
3:45 – 4:15	Hands-on: Comparing HashMap vs TreeMap Performance	30 min
4:15 – 4:30	Homework & Closing	15 min

1. HashMap

1.1 Definition

- Implements `Map` interface using a **hash table**.
- Stores key-value pairs: `HashMap<K, V>`.
- Allows one `null` key and multiple `null` values.
- **No ordering** – iteration order is not guaranteed (order may change over time).

1.2 Internal Working (Simplified)

- An array of **buckets** (`Node<K,V>[]`).
- `hashCode()` of key determines bucket index.
- If multiple keys land in same bucket, they are stored as a **linked list** (or **tree** after threshold for performance).
- `equals()` is used to compare keys within the same bucket.

1.3 Creating a HashMap

```
HashMap<String, Integer> map = new HashMap<>();
// With initial capacity (default 16) and load factor (default 0.75)
HashMap<String, Integer> map2 = new HashMap<>(32, 0.8f);
// From another map
```

```
HashMap<String, Integer> map3 = new HashMap<>(existingMap);
```

1.4 Important Methods

Method	Description
<code>put(K key, V value)</code>	Inserts or updates key-value pair. Returns previous value (or null).
<code>get(Object key)</code>	Returns value, or null if not present.
<code>getOrDefault(key, defaultValue)</code>	Returns value or default.
<code>remove(Object key)</code>	Removes mapping, returns value.
<code>containsKey(Object key)</code>	Checks key existence (O(1) average).
<code>containsValue(Object value)</code>	Checks value existence (O(n) average).
<code>keySet()</code>	Returns <code>Set<K></code> of all keys.
<code>values()</code>	Returns <code>Collection<V></code> of all values.
<code>entrySet()</code>	Returns <code>Set<Map.Entry<K,V>></code> of all entries.
<code>putIfAbsent(K key, V value)</code>	Puts only if key not already mapped (atomic).

1.5 Iteration over HashMap

```
// Iterate keys
for (String key : map.keySet()) { ... }

// Iterate values
for (Integer val : map.values()) { ... }

// Iterate entries (most efficient)
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    String k = entry.getKey();
    Integer v = entry.getValue();
}
```

1.6 Performance

- Average time for `put`, `get`, `remove` : **O(1)**.
- Worst case (many hash collisions): O(n) – but Java 8+ converts long lists to balanced trees (O(log n)).

1.7 When to Use HashMap?

1.7 WHEN TO USE HASHMAP:

- Fast lookups by key (dictionary, cache, phonebook).
- No need for sorted order.
- Keys have good `hashCode()` distribution.

2. TreeMap

2.1 Definition

- Implements `NavigableMap` (extends `SortedMap`) using a **Red-Black tree**.
- Keys are stored in **sorted order** (natural ordering or custom `Comparator`).
- No `null` keys (throws `NullPointerException`), but `null` values allowed.

2.2 Creating a TreeMap

```
// Natural ordering of keys (must implement Comparable)
TreeMap<String, Integer> treeMap = new TreeMap<>();

// With custom Comparator
TreeMap<String, Integer> customMap = new TreeMap<>(Comparator.reverseOrder());
```

2.3 Important Methods (in addition to HashMap)

Method	Description
<code>firstKey()</code> / <code>lastKey()</code>	Smallest / largest key.
<code>lowerKey(K key)</code>	Greatest key strictly less than given.
<code>higherKey(K key)</code>	Smallest key strictly greater.
<code>floorKey(K key)</code>	Greatest key $\leq$ given.
<code>ceilingKey(K key)</code>	Smallest key $\geq$ given.
<code>subMap(fromKey, toKey)</code>	View of portion.
<code>headMap(toKey)</code> / <code>tailMap(fromKey)</code>	Views.

2.4 Performance

- `put`, `get`, `remove` : $O(\log n)$ (tree operations).

2.5 When to Use TreeMap?

- Need keys in sorted order.
- Need range queries (submaps).
- Can accept slightly slower performance than HashMap.

3. HashSet

3.1 Definition

- Implements `Set` interface using a `HashMap` internally (dummy value).
- Stores **unique elements**, no duplicates (based on `hashCode()` and `equals()`).
- Allows one `null` element.
- **No ordering** – iteration order not guaranteed.

3.2 Creating a HashSet

```
HashSet<String> set = new HashSet<>();  
HashSet<Integer> numbers = new HashSet<>(initialCapacity);  
HashSet<String> copy = new HashSet<>(anotherCollection);
```

3.3 Important Methods

Method	Description
<code>add(E e)</code>	Adds if not present; returns true if added.
<code>remove(Object o)</code>	Removes element.
<code>contains(Object o)</code>	Checks existence (O(1) average).
<code>size()</code> , <code>isEmpty()</code> , <code>clear()</code>	Standard.
<code>iterator()</code>	Returns iterator.

3.4 When to Use HashSet?

- Need to eliminate duplicates.
- Fast membership testing (e.g., blacklist, visited set).
- Order does not matter.

4. TreeSet

4.1 Definition

- Implements `NavigableSet` (extends `SortedSet`) using a `TreeMap` internally.
- Elements sorted in natural order or by provided `Comparator`.
- No `null` allowed (throws `NullPointerException`).

4.2 Creating a TreeSet

```
TreeSet<Integer> sortedSet = new TreeSet<>();  
TreeSet<String> descending = new TreeSet<>(Comparator.reverseOrder());
```

4.3 Important Methods (similar to TreeMap keys)

Method	Description
<code>first()</code> , <code>last()</code>	Smallest / largest element.
<code>lower(E e)</code> , <code>higher(E e)</code>	Strictly less/greater.
<code>floor(E e)</code> , <code>ceiling(E e)</code>	$\leq$ or $\geq$.
<code>subSet(from, to)</code> , <code>headSet(to)</code> , <code>tailSet(from)</code>	Range views.

4.4 Performance

- `add`, `remove`, `contains`: $O(\log n)$.

4.5 When to Use TreeSet?

- Need unique sorted elements.
- Need range queries on the set.

5. Comparable vs Comparator

5.1 Need for Object Ordering

- `TreeMap` and `TreeSet` require keys/elements to be comparable.
- `Collections.sort()` and `Arrays.sort()` also require ordering.

5.2 Comparable (Natural Ordering)

- Interface: `java.lang.Comparable<T>`
- Method: `int compareTo(T o)`

- Returns negative, zero, positive if `this` is less, equal, greater than `o`.
- Implemented by classes like `Integer`, `String`, `Date`.
- **Single sorting sequence** – a class can have only one natural ordering.

5.3 Comparator (Custom Ordering)

- Interface: `java.util.Comparator<T>`
- Method: `int compare(T o1, T o2)`
- Can define multiple comparators for the same class.
- Useful for sorting on different fields (e.g., by name, then by age).
- Lambda expressions can implement Comparator easily.

5.4 Example Comparison Table

Feature	Comparable	Comparator
Package	<code>java.lang</code>	<code>java.util</code>
Method to implement	<code>compareTo()</code>	<code>compare()</code>
Parameter	single (other object)	two objects
Modifies class	Yes – changes the class	No – external
Multiple sequences	No	Yes
Used by	<code>TreeSet</code> , <code>TreeMap</code> , <code>Collections.sort(list)</code>	<code>Collections.sort(list, comparator)</code> , <code>TreeSet(comp)</code>

5.5 Rules for compareTo / compare

- Return negative if first < second, zero if equal, positive if first > second.
- Must be consistent with `equals()` (recommended but not required).
- Must be transitive, reflexive, and antisymmetric.

5.6 Example – Custom Class with Comparable

```
class Employee implements Comparable<Employee> {
    int id;
    String name;
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id); // natural order by id
    }
}
```

5.7 Example – Comparator using Lambda

```
Comparator<Employee> byName = (e1, e2) -> e1.name.compareTo(e2.name);
TreeSet<Employee> empSet = new TreeSet<>(byName);
```

6. LeetCode Problems – Approach Discussion

6.1 Group Anagrams

- **Problem:** Given array of strings, group anagrams together.
- **Key insight:** Two strings are anagrams if their sorted characters are equal.
- **Approach (HashMap):**
 - For each string, compute a key: sorted char array as String (or frequency hash).
 - Map key → list of original strings.
 - Return collection of values.
- **Complexities:** $O(n * k \log k)$ where k is max string length (sorting).
- **Alternative key:** frequency count using array of 26 chars → generates unique key as "1#2#0#..." .
- **OOP relevance:** `HashMap<String, List<String>>` is perfect here.

6.2 Top K Frequent Elements

- **Problem:** Given integer array, return k most frequent elements.
- **Approach 1 (HashMap + sorting):**
 - Build frequency map: `HashMap<Integer, Integer>` .
 - Convert to list of entries, sort by frequency descending, take first k .
 - Time: $O(n \log n)$.
- **Approach 2 (Bucket Sort):**
 - Create array of lists indexed by frequency ($\text{max frequency} \leq n$).
 - `Bucket[freq].add(num)`.
 - Then collect from highest bucket downwards.
 - Time: $O(n)$.
- **Approach 3 (Min-Heap):**
 - Use `PriorityQueue` with comparator on frequency, keep size k .
- **Key takeaway:** HashMap for counting, then heap or bucket for top k .

6.3 Majority Element

- **Problem:** Find element that appears more than $\lfloor n/2 \rfloor$ times.
 - **Approach 1 (HashMap):** Count frequencies, check if $> n/2$. $O(n)$ time, $O(n)$ space.
 - **Approach 2 (Boyer-Moore Voting Algorithm):**
 - Candidate = first element, count = 1.
 - Iterate: if same as candidate, count++ else count--. If count reaches 0, new candidate.
 - Final candidate is majority (guaranteed to exist per problem).
 - $O(n)$ time, $O(1)$ space.
 - **Edge cases:** Single element, all same, length 2.
-

7. HackerRank Problems – Approach Discussion

7.1 Java Map

- **Task:** Implement a phone book: store name $\rightarrow$ phone number mappings, then query for names.
- **Data structure:** `HashMap<String, String>` or `HashMap<String, Integer>`.
- **Input:** First integer n (entries), then n lines of "name phoneNumber", then query names until EOF.
- **Output:** For each query, if found print `name=phoneNumber`, else `Not found`.
- **Key learning:** Using `containsKey()` or `get()` with null check.

7.2 Java HashSet

- **Task:** Given pairs of strings (first, last), count number of unique pairs after each insertion.
 - **Data structure:** `HashSet<String>` where each element is `first + "," + last`.
 - **Output:** After each insertion, print current size of set.
 - **Key learning:** HashSet automatically handles duplicates if we create a unique string per pair.
-

8. Corporate Use Case: Fraud Detection & Transaction Monitoring System

8.1 Problem Context

A financial institution monitors millions of transactions daily to detect fraud. Requirements:

- Track **suspicious IP addresses** (blacklist) – fast membership check.
- Detect **frequent transactions from same user** within a short time window (e.g., > 10 transactions per minute) – need counting.
- Identify **commonly targeted merchants** – group transactions by merchant ID.

- Flag **duplicate transaction IDs** (possible replay attack) – use set.
- Generate **top-k suspicious accounts** based on transaction velocity.
- Maintain a **sorted view of high-value transactions** (above \$10,000) by amount.

8.2 Application of Day 7 Topics

Requirement	Java Feature Used
Blacklist of IPs	<code>HashSet<String></code> – O(1) contains check
Transaction count per user	<code>HashMap<String, Integer></code> (user → count)
Time-window counter (sliding)	<code>HashMap<String, Queue<Long>></code> (user → timestamps) – but counting uses map
Detect duplicate transaction IDs	<code>HashSet<String></code> – reject if already seen
Group transactions by merchant	<code>HashMap<String, List<Transaction>></code> or <code>HashMap<String, Integer></code> for count
Top-k suspicious users by transaction count	<code>HashMap</code> for counts, then min-heap (<code>PriorityQueue</code>) of top k
Sorted view of high-value transactions	<code>TreeSet<Transaction></code> with custom <code>Comparator</code> on amount
Find first transaction above threshold	<code>TreeSet.ceiling()</code> or <code>higher()</code>
Range queries: transactions between \$5000 and \$10000	<code>TreeSet.subSet()</code>
Natural ordering of Transaction by timestamp	Implement <code>Comparable<Transaction></code>
Custom ordering by amount for reporting	Comparator: <code>Comparator.comparing(Transaction::getAmount).reversed()</code>

8.3 Example Data Structures

```
// Blacklist
HashSet<String> blacklistedIPs = new HashSet<>();

// User transaction counts
```

```

HashMap<String, Integer> txCount = new HashMap<>();

// Duplicate transaction ID detection
HashSet<String> processedTxIds = new HashSet<>();

// Merchant transaction counts
HashMap<String, Integer> merchantTxCount = new HashMap<>();

// Sorted high-value transactions
TreeSet<Transaction> highValueTx = new TreeSet<>(
    Comparator.comparing(Transaction::getAmount)
                .thenComparing(Transaction::getTimestamp)
);

// Top-k suspicious users (using priority queue)
PriorityQueue<Map.Entry<String, Integer>> topK = new PriorityQueue<>(
    (a,b) -> a.getValue() - b.getValue() // min-heap by count
);

```

8.4 Business Logic Examples

- **Fraud rule:** If a user does > 50 transactions in 1 minute → flag.
 - Use `HashMap<String, Queue<Long>>` to store timestamps; remove older than 60 seconds.
- **Duplicate transaction:** If `txId` already in `processedTxIds`, reject.
- **Merchant targeted:** If merchant gets > 1000 transactions in 1 hour, investigate.
- **High-value monitoring:** Automatically add transaction to `TreeSet` if amount > 10000. Then report top 5 highest.

8.5 Why HashMap for Counts?

- Fast increment: `map.put(user, map.getOrDefault(user, 0) + 1);`
- `TreeMap` would be $O(\log n)$ and slower for large counts.

8.6 Discussion Questions

- How would you implement a time-bounded counter (e.g., last 5 minutes only)? (Use a queue per user or a sliding window map.)
- What happens if we use `TreeMap` instead of `HashMap` for user counts? (Slower, but could be used if we need sorted users by name – not needed here.)
- How to ensure thread-safety for these collections in a real-time system? (Use `ConcurrentHashMap` and `ConcurrentSkipListSet`.)
- Why use `TreeSet` for high-value transactions? (Need sorted order and range queries – e.g., find all > 100000.)

9 Hands-on Exercises (Without Code Lines)

9. HANDS-ON EXERCISES (WITHOUT CODE LINES)

9.1 HashMap – Frequency Counter

Describe the steps to count word frequencies in a sentence using `HashMap<String, Integer>` .

9.2 HashSet – Remove Duplicates

Given a list of integers with duplicates, explain how to obtain a list of unique elements preserving first occurrence order. (Hint: Use `LinkedHashSet` but not covered; or use `HashSet` plus `ArrayList` but order not preserved; better: iterate and add to set, if not present add to result list.)

9.3 TreeMap – First and Last Keys

If `TreeMap<String, Integer> grades = new TreeMap<>();` with keys "Alice", "Bob", "Charlie", what do `firstKey()` and `lastKey()` return? (Alice, Charlie)

9.4 Comparable vs Comparator

For a `Student` class with `rollNo` and `name` , describe:

- How to sort by `rollNo` using `Comparable` .
- How to sort by `name` using `Comparator` .

9.5 Group Anagrams – Key Generation

For the string "eat", what are two possible keys to group with "tea" and "ate"? (Sorted "aet" or frequency string "1#0#0#...")

9.6 Majority Element – Dry Run

Run Boyer-Moore algorithm on array: [2,2,1,1,1,2,2]. Show candidate and count changes.

10. Quiz & Quick Check (Conceptual)

1. Which of the following allows `null` keys?

- (a) `TreeMap`
- (b) `HashMap`
- (c) `TreeSet`
- (d) Both a and c

2. Time complexity of `get()` operation in a `HashMap` with good hash function is

2. Time complexity of `get()` operation in a `HashMap` with good hash function is:

- (a) $O(\log n)$
- (b) $O(n)$
- (c) $O(1)$ average
- (d) $O(n \log n)$

3. To sort a list of custom objects by multiple fields, you should use:

- (a) `Comparable` only
- (b) `Comparator`
- (c) Neither
- (d) Both are same

4. What does `HashSet` use internally to store elements?

- (a) `TreeMap`
- (b) `HashMap`
- (c) `ArrayList`
- (d) `LinkedList`

5. In the Boyer-Moore majority element algorithm, if a majority element exists, it will be:

- (a) The first element
- (b) The last element
- (c) The candidate after one pass
- (d) None of the above

Answers: 1-b, 2-c, 3-b, 4-b, 5-c

11. Additional Resources & Cheat Sheets

- **HashMap vs TreeMap vs LinkedHashMap** comparison table.
- **hashCode and equals contract** – essential for HashMap/HashSet.
- **Boyer-Moore voting algorithm** – step-by-step example.
- **Comparator factory methods:** `Comparator.comparing()`, `thenComparing()`, `reversed()`.

12. Closing Summary

Day 7 completed the Collections Framework with maps and sets, essential for fast lookups, counting, and uniqueness. Learners explored `HashMap` and `HashSet` for $O(1)$ average operations, `TreeMap` and `TreeSet` for sorted order, and the difference between `Comparable` and `Comparator`.

operations, `TreeMap` and `TreeSet` for sorted order, and the difference between `Comparable` and `Comparator`. LeetCode problems demonstrated real-world applications: grouping anagrams, finding top k frequent elements, and detecting majority elements. The Fraud Detection case study integrated these concepts into a financial monitoring system, detecting duplicate transactions, blacklisted IPs, and high-value anomalies.

Key Takeaways:

- `HashMap` is the go-to for key-value storage with fast access.
- `TreeMap` maintains sorted keys at $O(\log n)$ cost.
- `HashSet` provides uniqueness and fast membership.
- `TreeSet` provides sorted unique elements.
- `Comparable` for natural ordering; `Comparator` for multiple custom sorts.

End of Day 7